DBMS – WEEK 6 REPORT

PRODUCT REVIEW AND RATING MANAGEMENT SYSTEM

SCENARIO

The stationery shop system needs to collect customer feedback about purchased products.
Customers can provide ratings and reviews based on their product experience.
The system stores these reviews and ratings to analyze product performance.
It also helps identify highly rated products and customer feedback.

TABLE CREATION

A **Review** table was created to store customer reviews, product ratings, comments and review dates.

```
SQL> CREATE TABLE Review (REVIEW_ID VARCHAR2(10) PRIMARY KEY, CUSTOMER_ID INT NOT NULL, PRODUCT_ID VARCHAR(7) NOT NULL, RATING NUMBER(1) CHECK (RATI
NG BETWEEN 1 AND 5), REVIEW_COMMENT VARCHAR2(200), REVIEW_DATE DATE, CONSTRAINT FK_REVIEW_CUSTOMER FOREIGN KEY (CUSTOMER_ID) REFERENCES Customer(CUS
TOMER_ID), CONSTRAINT FK_REVIEW_PRODUCT FOREIGN KEY (PRODUCT_ID) REFERENCES Product(PRODUCT_ID));

Table created.
```

RELATIONSHIP

The **Review** table is connected with the existing **Customer** and **Product** tables.

- A customer can provide reviews for purchased products.
- A product can receive reviews from different customers.
- Customer_ID connects **Customer** with **Review**.
- Product_ID connects **Product** with **Review**.

This relationship allows customer feedback to be associated with the corresponding product.

INSERTING REVIEW DATA

Sample customer reviews and ratings were inserted into the Review table.

```
SQL> INSERT INTO Review VALUES ('R001',101,'P001',4,'Good quality and useful',TO_DATE('15-SEP-2026','DD-MON-YYYY'));

1 row created.

SQL> INSERT INTO Review VALUES ('R002',102,'P002',5,'Excellent and worth the price',TO_DATE('18-SEP-2026','DD-MON-YYYY'));

1 row created.

SQL> INSERT INTO Review VALUES ('R003',103,'P003',3,'Product is satisfactory',TO_DATE('20-SEP-2026','DD-MON-YYYY'));

1 row created.

SQL> INSERT INTO Review VALUES ('R004',104,'P004',4,'Nice product with good quality',TO_DATE('25-SEP-2026','DD-MON-YYYY'));

1 row created.
```

DISPLAY ALL REVIEWS

```
SQL> SELECT * FROM Review;

REVIEW_ID  CUSTOMER_ID PRODUCT_ID RATING REVIEW_COMMENT
---------- ----------- ---------- ------ ----------------------------------
REVIEW_DATE
------------
R001               101 P001            4 Good quality and useful
15-SEP-26

R002               102 P002            5 Excellent and worth the price
18-SEP-26

R003               103 P003            3 Product is satisfactory
20-SEP-26


REVIEW_ID  CUSTOMER_ID PRODUCT_ID RATING REVIEW_COMMENT
---------- ----------- ---------- ------ ----------------------------------
REVIEW_DATE
------------
R004               104 P004            4 Nice product with good quality
25-SEP-26
```

UPDATE REVIEW COMMENT

```
SQL> UPDATE Review SET REVIEW_COMMENT='Very good product' WHERE REVIEW_ID='R003';

1 row updated.
```

DELETE REVIEW

```
SQL> DELETE FROM Review WHERE REVIEW_ID='R004';

1 row deleted.
```

RETRIEVE PRODUCT REVIEW DETAIL

1. Display all reviews for a product.

```
SQL> SELECT * FROM Review WHERE PRODUCT_ID='P001';

REVIEW_ID  CUSTOMER_ID PRODUCT_ID RATING REVIEW_COMMENT
---------- ----------- ---------- ------ --------------------------
REVIEW_DATE
-----------
R001                101 P001            4 Good quality and useful
15-SEP-26
```

2. Display customer name with their reviews.

```
SQL> SELECT C.CUSTOMER_NAME,R.REVIEW_COMMENT,R.RATING FROM Customer C JOIN Review R ON C.CUSTOMER_ID=R.CUSTOMER_ID;

CUSTOMER_NAME
--------------------------------------------
REVIEW_COMMENT                     RATING
---------------------------------- ------
Arun Kumar Raj
Good quality and useful                 4

Priya Sharma
Excellent and worth the price           5

Rahul Raj
Very good product                       3
```

3. Find products having maximum reviews.

```
SQL> SELECT PRODUCT_ID,COUNT(*) AS REVIEW_COUNT FROM Review GROUP BY PRODUCT_ID ORDER BY REVIEW_COUNT DESC;

PRODUCT_ID REVIEW_COUNT
---------- ------------
P001                1
P002                1
P003                1
```

4. Display recent customer feedback.

```
SQL> SELECT * FROM Review ORDER BY REVIEW_DATE DESC;

REVIEW_ID  CUSTOMER_ID PRODUCT_ID RATING REVIEW_COMMENT
---------- ----------- ---------- ------ ------------------------------
REVIEW_DATE
------------
R003               103 P003            3 Very good product
20-SEP-26

R002               102 P002            5 Excellent and worth the price
18-SEP-26

R001               101 P001            4 Good quality and useful
15-SEP-26
```

5. Retrieve reviews with ratings above 4.

```
SQL> SELECT * FROM Review WHERE RATING>4;

REVIEW_ID  CUSTOMER_ID PRODUCT_ID RATING REVIEW_COMMENT
---------- ----------- ---------- ------ ------------------------------
REVIEW_DATE
------------
R002               102 P002            5 Excellent and worth the price
18-SEP-26
```

CALCULATE AVERAGE PRODUCT RATINGS

1. Calculate average rating for each product.

```
SQL> SELECT PRODUCT_ID,AVG(RATING) AS AVERAGE_RATING FROM Review GROUP BY PRODUCT_ID;

PRODUCT_ID AVERAGE_RATING
---------- --------------
P001                    4
P003                    3
P002                    5
```

2. Count the number of reviews per product

```
SQL> SELECT PRODUCT_ID,COUNT(REVIEW_ID) AS NUMBER_OF_REVIEWS FROM Review GROUP BY PRODUCT_ID;

PRODUCT_ID NUMBER_OF_REVIEWS
---------- -----------------
P001                       1
P003                       1
P002                       1
```

.

3. Find highest-rated products.

```
SQL> SELECT PRODUCT_ID,AVG(RATING) AS AVERAGE_RATING FROM Review GROUP BY PRODUCT_ID ORDER BY AVERAGE_RATING DESC;

PRODUCT_ID AVERAGE_RATING
---------- --------------
P002                    5
P001                    4
P003                    3
```

4. Identify products with average rating above 4.

```
SQL> SELECT PRODUCT_ID,AVG(RATING) AS AVERAGE_RATING FROM Review GROUP BY PRODUCT_ID HAVING AVG(RATING)>4;

PRODUCT_ID AVERAGE_RATING
---------- --------------
P002                    5
```

IDENTIFY HIGHLY RATED PRODUCT

Report 1: Product Rating Analysis

Display:

- Product name
- Number of reviews
- Average rating

```
SQL> SELECT P.PRODUCT_NAME,COUNT(R.REVIEW_ID) AS NUMBER_OF_REVIEWS,ROUND(AVG(R.RATING),2) AS AVERAGE_RATING FROM Product P LEFT JOIN
Review R ON P.PRODUCT_ID=R.PRODUCT_ID GROUP BY P.PRODUCT_NAME;

PRODUCT_NAME
--------------------------------------------------------------------------------
NUMBER_OF_REVIEWS AVERAGE_RATING
----------------- --------------
Ball Pen
                1              4

Notebook
                0

Pencil Box
                0


PRODUCT_NAME
--------------------------------------------------------------------------------
NUMBER_OF_REVIEWS AVERAGE_RATING
----------------- --------------
Gel Pen
                1              5

Spiral Notebook
                0

Pencil
                1              3


6 rows selected.
```

Report 2: Customer Feedback Analysis

Find:

- Most reviewed products

```
SQL> SELECT P.PRODUCT_NAME,COUNT(R.REVIEW_ID) AS NUMBER_OF_REVIEWS FROM Product P JOIN Review R ON P.PRODUCT_ID=R.PRODUCT_ID GROUP BY
 P.PRODUCT_NAME ORDER BY NUMBER_OF_REVIEWS DESC;

PRODUCT_NAME
--------------------------------------------------------------------------------
NUMBER_OF_REVIEWS
-----------------
Ball Pen
                1

Pencil
                1

Gel Pen
                1
```

- Highly rated products

```
SQL> SELECT P.PRODUCT_NAME,ROUND(AVG(R.RATING),2) AS AVERAGE_RATING FROM Product P JOIN Review R ON P.PRODUCT_ID=R.PRODUCT_ID GROUP B
Y P.PRODUCT_NAME HAVING AVG(R.RATING)>4;

PRODUCT_NAME
--------------------------------------------------------------------------------
AVERAGE_RATING
--------------
Gel Pen
             5
```

- Products requiring improvement

```
SQL> SELECT P.PRODUCT_NAME,ROUND(AVG(R.RATING),2) AS AVERAGE_RATING FROM Product P JOIN Review R ON P.PRODUCT_ID=R.PRODUCT_ID GROUP B
Y P.PRODUCT_NAME HAVING AVG(R.RATING)<3;

no rows selected
```

Report 3: Rating Distribution

- Number of 5-star ratings

```
SQL> SELECT COUNT(*) AS FIVE_STAR_RATINGS FROM Review WHERE RATING=5;

FIVE_STAR_RATINGS
-----------------
                1
```

- Number of 4-star ratings

```
SQL> SELECT COUNT(*) AS FOUR_STAR_RATINGS FROM Review WHERE RATING=4;

FOUR_STAR_RATINGS
-----------------
                1
```

- Number of low-rated products

```
SQL> SELECT COUNT(*) AS LOW_RATED_REVIEWS FROM Review WHERE RATING<=2;

LOW_RATED_REVIEWS
-----------------
                0
```

- Complete Rating

```
SQL> SELECT RATING,COUNT(*) AS NUMBER_OF_RATINGS FROM Review GROUP BY RATING ORDER BY RATING DESC;

RATING NUMBER_OF_RATINGS
------ -----------------
     5                 1
     4                 1
     3                 1
```